Informe de pruebas unitarias y cobertura de NutriVida

Aplicación React de la segunda entrega

Asignatura: Desarrollo Full Stack II — DSY1104
Docente: Hernán Saavedra
Equipo: Jonathan Rivera, Nitsi Sepúlveda y Sofía González
Fecha de entrega: 17 de octubre de 2026

1. Propósito y resultado general

Este informe documenta el proceso de pruebas automatizadas de NutriVida, su configuración, los comportamientos verificados y el análisis de cobertura. El prototipo frontend incluye registro, sesión simulada, catálogo por categorías, ofertas, agendamiento, revisión y confirmación de reservas y administración de pacientes y profesionales.

La ejecución de referencia con cobertura comenzó el 16 de octubre de 2026 a las 02:20:55, hora de Chile. Vitest informó 151 pruebas aprobadas en 15 archivos, sin pruebas fallidas, con una duración de 34,07 segundos. La compilación de producción de esta versión se había verificado previamente y finalizó correctamente.

Indicador	Resultado
Archivos aprobados	15 de 15
Casos aprobados	151 de 151
Sentencias ejecutadas	90,29 %
Ramas ejecutadas	81,89 %
Funciones ejecutadas	93,44 %
Líneas ejecutadas	90,20 %

Entorno utilizado

La ejecución se realizó en Windows mediante Git Bash. El proyecto utiliza Node.js 24, React, React Router, React Bootstrap y React Hook Form. Se emplearon Vitest 5.0.4, @vitest/coverage-v8 5.0.4, React Testing Library, user-event, jest-dom y jsdom. La compilación registrada utilizó Vite 8.3.2.

Las versiones de dependencias se conservan en package-lock.json. La evidencia no incluye la versión menor exacta de Node.js ni la versión de npm; no se atribuyen valores que no hayan sido registrados.

2. Alcance y configuración del proceso

Las pruebas cubren funciones auxiliares, contextos, componentes interactivos y navegación entre páginas. Algunos casos comprueban una unidad aislada; otros integran componentes con contextos y rutas para verificar flujos de interfaz. Se incluyen caminos válidos, entradas inválidas y fallos simulados del almacenamiento.

Configuración de Vitest

vite.config.js define jsdom como entorno de ejecución, habilita las API globales de Vitest y carga src/setupTests.js. Ese archivo incorpora jest-dom para realizar verificaciones sobre el DOM, como presencia, valores de controles, atributos accesibles y mensajes visibles.

La cobertura utiliza el proveedor v8 y genera tres salidas: resumen de terminal, reporte HTML y coverage-summary.json. La carpeta de generación es coverage/. La medición incluye src/**/*.{js,jsx}, incluso archivos sin pruebas directas. Se excluyen los archivos de pruebas, setupTests.js y main.jsx, que monta la aplicación en el navegador.

Diseño de los casos

Cada caso prepara los datos y el entorno, ejecuta una acción y comprueba el resultado esperado. user-event reproduce acciones como escribir, pegar texto, seleccionar opciones y pulsar botones. Las consultas por etiqueta, rol y nombre accesible permiten verificar la interfaz desde la perspectiva de uso.

Se utilizan pruebas parametrizadas para aplicar las mismas verificaciones a varias entradas, como dominios de correo, RUN y fechas. Cada entrada ejecutada representa un caso independiente. El número de archivos no equivale al número de pruebas.

Independencia de las pruebas

Los casos que utilizan almacenamiento preparan sus datos de demostración y limpian localStorage según el entorno de cada archivo. cleanup desmonta los componentes y vi.restoreAllMocks restaura las funciones intervenidas. Las pruebas de fechas fijan el reloj y recuperan los temporizadores reales al terminar.

Límites del alcance

jsdom representa el DOM, pero no sustituye un navegador real para evaluar distribución visual, reproducción de videos o comportamiento de un servidor publicado. Las pruebas no verifican pagos, envío de correos ni autenticación de servidor, porque esos servicios no forman parte del prototipo. La revisión responsive se realizó manualmente y es una verificación complementaria.

3. Casos representativos y trazabilidad

Los siguientes diez casos resumen comportamientos comprobados por la suite. Son una selección para explicar el proceso; no representan el total de 151 casos ejecutados. Los archivos indicados se encuentran dentro de src/.

Caso y archivo	Acción y resultado verificado
01 Registro pages/Registro.test.jsx	Enviar el formulario vacío muestra errores obligatorios e impide llamar a registrarPaciente.
02 Registro válido pages/Registro.test.jsx	Ingresar datos válidos con un dominio admitido envía los datos normalizados y navega al inicio de sesión.
03 Sesión de paciente pages/Login.test.jsx	Ingresar credenciales válidas crea la sesión local y navega al inicio público.
04 Acceso por rol pages/Login.test.jsx	Un paciente que usa el acceso administrativo recibe un error y no obtiene una sesión administrativa.
05 Contacto components/FormularioContacto.test.jsx	Un envío válido muestra el carácter simulado de la operación y limpia los campos.
06 Agendamiento pages/Agendar.test.jsx	Una fecha pasada o un horario duplicado para el mismo profesional, fecha y hora bloquean el agendamiento.
07 Categorías y ofertas pages/Catalogo.test.jsx	Filtrar categorías actualiza los planes y la URL. Las ofertas muestran condiciones y dirigen al agendamiento con el plan correspondiente.
08 Revisión del carrito pages/Carrito.test.jsx	Revisar consultas no crea citas. Se validan los datos del paciente y se confirma conservando el precio del plan; eliminar una consulta actualiza el total.
09 Confirmación con error context/CarritoContext.test.jsx	Un fallo al guardar citas impide completar la confirmación y conserva las reservas.
10 Administración pages/Admin.test.jsx	Crear un nutricionista desde el formulario guarda sus datos sin exigir un precio adicional por profesional.

Los resultados de aprobación corresponden a la ejecución completa del 16 de octubre de 2026 a las 02:20:55. Las verificaciones utilizan aserciones sobre el DOM, las respuestas del contexto, los parámetros de navegación y los datos almacenados.

4. Organización de las pruebas y mocks

Archivo dentro de src	Responsabilidad principal
App.test.jsx	Rutas públicas, composición y acceso al panel.
components/Navbar.test.jsx	Opciones por sesión, contador y navegación.
components/ScrollNavegacion.test.jsx	Desplazamiento, hash y cancelación de animación.
components/FormularioContacto.test.jsx	Validaciones, mensajes y limpieza del formulario.
components/ListaNutricionistas.test.jsx	Datos de profesionales y apertura y cierre del detalle.
components/PlanesServicios.test.jsx	Planes, promoción y selección mediante la URL.
context/AuthContext.test.jsx	Registro, duplicados, sesión y conservación de datos.
context/CarritoContext.test.jsx	Persistencia, validación, fallos y reintentos.
pages/Admin.test.jsx	Acceso, búsquedas y operaciones administrativas.
pages/Agendar.test.jsx	Preselección, fecha y disponibilidad.
pages/Carrito.test.jsx pages/Catalogo.test.jsx	Carrito: revisión, datos, total y confirmación. Catálogo: filtros, ofertas y navegación.
pages/Login.test.jsx	Interacción, accesos, navegación y errores.
pages/Registro.test.jsx	Datos personales, dominios, comunas y envío.
utils/utils.test.js	Almacenamiento, fechas, RUN y formatos.

Uso de mocks y controles del entorno

En Registro se proporciona un AuthContext controlado con vi.fn para responder con éxito, duplicados o un fallo de almacenamiento. Esto permite probar la reacción del formulario de forma aislada. Las pruebas de Login utilizan el AuthProvider real para comprobar la integración con la sesión local.

vi.spyOn interviene Storage.prototype.getItem, setItem o removeItem para provocar fallos de lectura, escritura y eliminación. Se verifica que la operación informe el error y conserve los datos en los escenarios probados. console.error se silencia únicamente en los casos que provocan esos errores de forma deliberada.

Los espías de scrollTo, requestAnimationFrame, cancelAnimationFrame y scrollIntoView permiten observar el desplazamiento sin depender de una ventana real. Las pruebas de fechas usan vi.useFakeTimers y vi.setSystemTime. MemoryRouter controla rutas en pruebas de componentes; App se prueba con su BrowserRouter propio, sin anidar otro router.

5. Resultados y análisis de cobertura

Métrica	Cubierto	Total	Cobertura
Sentencias	791	876	90,29 %
Ramas	638	779	81,89 %
Funciones	228	244	93,44 %
Líneas	764	847	90,20 %

Las sentencias y líneas indican las instrucciones ejecutadas; las funciones muestran cuántas se invocaron y las ramas reflejan alternativas de decisión recorridas. La cobertura permite localizar código sin ejecutar, pero una instrucción ejecutada no garantiza que todas sus condiciones y resultados hayan sido comprobados.

Área	Sentencias	Ramas	Funciones	Líneas
src	100 %	100 %	100 %	100 %
components	93,58 %	88,18 %	93,15 %	94,11 %
context	84,52 %	72,20 %	90,14 %	84,39 %
data	100 %	100 %	100 %	100 %
pages	92,65 %	85,38 %	95,34 %	92,61 %
utils	100 %	91,30 %	100 %	100 %

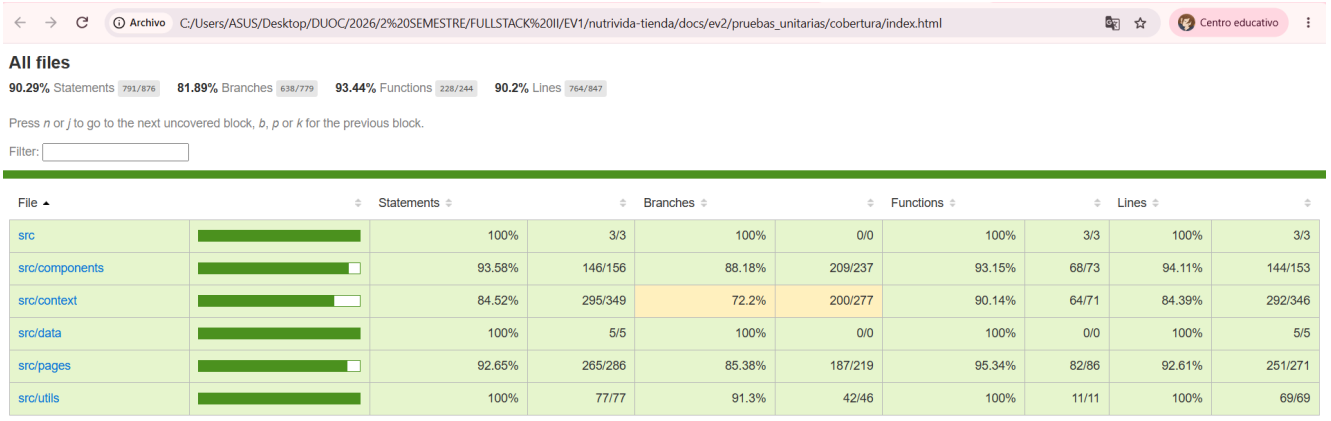


Figura 1. Resumen de cobertura de la ejecución del 16 de octubre de 2026, generado con Vitest y el proveedor v8.

Interpretación de los resultados

La suite ejercita las principales interacciones y reglas del prototipo. Los contextos concentran más decisiones alternativas por las validaciones, relaciones entre registros y manejo de errores de almacenamiento. Por ello, su cobertura de ramas es inferior a la de los componentes de presentación.

DatosContext registra 65,04 % de ramas y 78,94 % de líneas. AdminHome registra 20 % de ramas y Ofertas, 50 %: esta última tiene pendiente el camino alternativo sin un plan asociado a la promoción. Servicios alcanza 100 % en las cuatro métricas; Carrito alcanza 92,30 % de sentencias y 92,64 % de ramas. El reporte detallado identifica las líneas y decisiones que no se ejecutaron.

Como ampliación del proceso, se priorizarían escenarios de estadísticas con distintas citas, operaciones administrativas alternativas y errores de recuperación o eliminación de sesión. Los componentes estáticos con 100 % de cobertura requieren revisión visual adicional; ese valor no acredita por sí solo accesibilidad completa ni corrección de todo el contenido.

Correcciones durante el proceso

Una ejecución previa con cobertura excedió los 5 segundos en dos casos de Registro al completar el formulario carácter por carácter. Se optimizó el helper de prueba para pegar texto mediante user-event, manteniendo la escritura de fechas, las selecciones y las aserciones. No se aumentó el timeout ni se eliminaron casos. La última ejecución aprobó la suite completa.

Las pruebas del menú motivaron el ajuste del nombre accesible del botón mediante label y la exposición de aria-expanded. En Carrito se corrigió el selector de la prueba para consultar el rol checkbox realmente expuesto por React Bootstrap. En el catálogo se precisó la consulta del contador por su texto para distinguirlo del output de navegación, que también tiene rol status. Se conservaron las aserciones funcionales y los casos de prueba.

6. Ejecución y evidencias

Desde la raíz del repositorio, ingresar a fase2/proyecto_etapa2_react. Los comandos de npm se ejecutan en esa carpeta, donde está package.json. Para reproducir el entorno se utiliza npm ci con el archivo de bloqueo incluido en el repositorio.

Comando	Propósito
npm ci	Instalar las dependencias registradas.
npm run test -- --run	Ejecutar toda la suite una vez.
npm run test:coverage	Ejecutar las pruebas y generar la cobertura.
npm run build	Generar la compilación de producción.
npm run preview	Revisar la compilación en un servidor local.

El reporte generado se abre en coverage/index.html con un navegador. Para conservar la evidencia en la documentación, se copia el contenido completo de coverage/ a docs/ev2/pruebas_unitarias/cobertura/, manteniendo sus recursos CSS y JavaScript. La captura de resumen se actualiza en evidencias/cobertura-final.png.

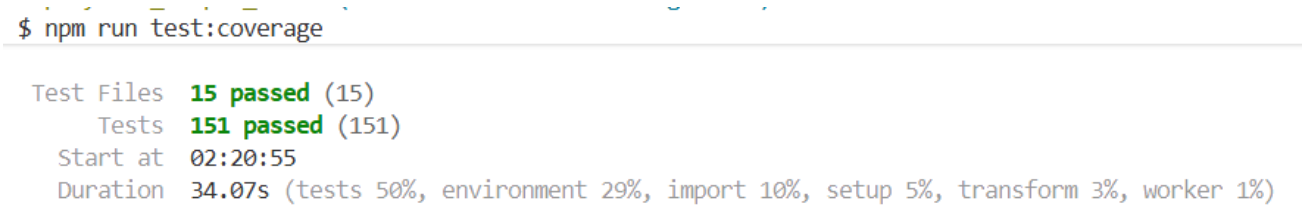


Figura 2. Resultado de npm run test:coverage del 16 de octubre de 2026: 151 pruebas aprobadas en 15 archivos, con una duración de 34,07 segundos.

Ubicación de los entregables

Ubicación desde la raíz	Contenido
fase2/proyecto_etapa2_react/src/	Archivos ejecutables de pruebas junto al código.
fase2/proyecto_etapa2_react/vite.config.js	Configuración del entorno y de la cobertura.
docs/ev2/pruebas_unitarias/cobertura/index.html	Entrada del reporte navegable de cobertura.
docs/ev2/pruebas_unitarias/cobertura/coverage-summary.json	Resultados numéricos generados por v8.

docs/ev2/pruebas_unitarias/evidencias/cobertura-final.png	Captura del resumen de cobertura.
docs/ev2/pruebas_unitarias/Informe Pruebas NutriVida.pdf	Informe de alcance, proceso y resultados.
docs/ev2/DSY1104 Evaluación Parcial 2 - Anexo 4 ERS-Especificacion de Requisitos del software.docx	Especificación de Requisitos de Software versión 2, actualizada para la aplicación React y el alcance de la segunda entrega.

Conclusiones

La ejecución documentada aprobó 151 pruebas en 15 archivos. La compilación de producción de esta versión también se verificó sin errores. Los casos aportan evidencia sobre formularios, navegación, acceso por rol, administración, categorías, ofertas, revisión de datos del paciente, precio del plan y conservación de reservas ante fallos simulados.

La cobertura obtenida permite identificar decisiones que todavía requieren escenarios adicionales. Los resultados respaldan los comportamientos comprobados dentro del alcance frontend; no certifican ausencia de defectos ni sustituyen la revisión manual del diseño, la reproducción del video o la comprobación del despliegue en AWS.

Fuentes del informe

Repositorio: https://github.com/soofigonb/nutrivida-tienda-online
Registro de npm run test:coverage del 16 de octubre de 2026 a las 02:20:55, hora de Chile: 151 pruebas aprobadas y resumen de cobertura v8.
Instrucciones de la Evaluación Parcial 2 de DSY1104 y guía docente de pruebas con React, Vite y Vitest.